

ADI TFLITE MICRO SHARCFX USER'S GUIDE

ANALOG DEVICES, INC.

www.analog.com

(4.0.0, July 2026)

Table of Contents

1 Introduction	5
1.1 Purpose	5
1.2 Scope of this Manual	5
1.3 Acronyms.....	5
1.4 Additional Information	5
2 Product Overview.....	6
2.1 Software System Overview	6
3 Setup, Install, Build and Run.....	6
3.1 Hardware Setup.....	6
3.1.1 Switches and Jumpers	6
3.1.2 Required Hardware	6
3.1.3 Setting up the Hardware for EV-SC835-SOM EZ-KIT® REV A	7
3.2 Instructions to build and run example application on CCES	8
3.2.1 To build example projects and run	8
3.2.2 Running the File I/O Application.....	9
3.2.1 Running the Realtime Application	9
3.3 Instructions to build the TFLM library.	9
3.4 Headless Build and Flash Instructions	10
3.4.1 Build Instructions.....	10
3.4.2 Flash Instructions.....	11
3.5 Specifications for current reference applications	12
3.6 UART Setup	12
3.7 Running your own model (For advanced users).....	13
4 Flashing application on board using CCES	14
4.1 Changes to be made in the target application.....	14
4.1.1 Ensure boot mode is set to SPI and boot format is set to Include.....	14
4.1.2 Defining the initialization file	15
4.1.3 Build target project with these additional settings	15
4.2 Flashing the generated loader file onto the board	16
4.2.1 Copy the generated loader file into Flash Programmer project.....	16
4.2.2 Bring in the loader file into CCES project	16
4.2.3 Build and Run the Flash Programmer	16
4.2.4 Using flash-able prebuilt loader files.....	17

5 Multi-Boot Functionality (ADSP-SC83x SPI Flash)	17
5.1 Generation Loader file	17
5.2 Flashing Overview	18
6 Weights reordering	19
6.1 Reordering layer weights in the model	20
6.2 Enabling new kernels.....	20

List of Figures

Figure 1: ADSP- SC835 board connections.....	7
Figure 2: ADSP- SC835 audio connections	9

Copyright, Disclaimer & Trademark Statements

Copyright Information

Copyright (c) 2024 Analog Devices, Inc. All Rights Reserved. This documentation is proprietary and confidential to Analog Devices, Inc. and its licensors. This document may not be reproduced in any form without prior, express consent from Analog Devices, Inc.

Disclaimer

Analog Devices, Inc. (Analog Devices) reserves the right to change this product without prior notice. Information furnished by Analog Devices is believed to be accurate and reliable. However, no responsibility is assumed by Analog Devices for its use; nor for any infringement of patents or other rights of third parties which may result from its use. No license is granted by implication or otherwise under the patent or other rights of Analog Devices

Trademark and Service Mark Notice

Analog Devices, the Analog Devices logo, Blackfin, SHARC, TigerSHARC, CrossCore®, VisualDSP, VisualDSP++, EZ-KIT Lite, EZ-Extender, SigmaStudio and Collaborative are the exclusive trademarks and/or registered trademarks of Analog Devices, Inc (Analog Devices).

All other brand and product names are trademarks or service marks of their respective owners.

Analog Devices' Trademarks and Service Marks may not be used without the express written consent of Analog Devices, such consent only to be provided in a separate written agreement signed by Analog Devices. Subject to the foregoing, such Trademarks and Service Marks must be used according to Analog Devices' Trademark Usage guidelines. Any licensee wishing to use Analog Devices' Trademarks and Service Marks must obtain and follow these guidelines for the specific marks at issue.

1 Introduction

1.1 Purpose

This document describes how to build and run the reference applications for ADI TFLite Micro SharcFX module on the SharcFX platform.

1.2 Scope of this Manual

This document describes how to work with ADI TFLite Micro SharcFX on the EV-SC835-SOM EZ-KIT® REV A board. This document explains how to build and run the applications that accompany this product.

This document is intended for developers who are familiar with the TensorFlow Lite for Micro APIs and have familiarity with converting their Tensorflow models to TFLite. This product has some reference applications which serve as a reference on how to build and run your applications with the SharcFX TFLite Micro library. This document assumes a background in ADI's Sharc family of processors.

1.3 Acronyms

ADI	Analog Devices Inc.
EZ-KIT	EZ-KIT Lite or EZ-Board
ICE	In Circuit Emulator
JTAG	Joint Test Action Group
USB	Universal Serial Bus
CCES	CrossCore® Embedded Studio
GUI	Graphical User Interface

1.4 Additional Information

For more information on the latest ADI processors, silicon errata, code examples, development tools, system services and devices drivers, technical support and any other additional information, please visit our website at <http://www.analog.com/processors>.

2 Product Overview

2.1 Software System Overview

The TFLite Micro Library for SharcFX is built upon the TensorFlow Lite Micro framework developed by Google. You can read more about this framework [here](#). Our optimized implementation of this framework is designed to specifically run on ADI's SharcFX line of processors.

The product consists of the TFLite Micro library for SharcFX, sample applications and associated documents.

3 Setup, Install, Build and Run

At the time of release, this product was tested with CrossCore® Embedded Studio version 3.0.2 on a Windows 10 based host machine. Please note that applications using this product should be able to build and run using any future releases of the tool chain and any OS that it supports. For information about the CrossCore® Embedded Studio tool chain refer to www.analog.com/cces.

3.1 Hardware Setup

3.1.1 Switches and Jumpers

None

3.1.2 Required Hardware

EV-SC835-SOM EZ-KIT® REV A Projects' hardware requirements

- 1) Analog Devices EV-SC835-SOM EZ-KIT® Rev A
- 2) Analog Devices EV-SOMCRR-EZ-KIT® Rev D
- 2) ICE-1000/ ICE-2000
- 3) USB cable with USB A and Micro USB-B connectors (for ICE connection)
- 4) USB cable with USB A and Type C connectors (for board connection)
- 5) 12 V AC adaptor to Power EV-SOMCRR-EZ-KIT® REV D board

3.1.3 Setting up the Hardware for EV-SC835-SOM EZ-KIT® REV A

Set up the hardware as shown in the figure below:

- 1) Connect the EV-SC835-SOM EZ-KIT® REV A board to the EV-SOMCRR-EZ-KIT® REV D as in the Figure 1.
- 2) Connect the ICE-1000 or ICE-2000 to the 'JTAG' connection (Refer to Figure 1) and the other end to one of the PC's USB ports.
- 3) Connect the USB Type C connector to the EV-SC835-SOM EZ-KIT® REV A board (refer to Figure 1) and the other end to one of the PC's USB ports.
- 4) Power the EV-SOMCRR-EZ-KIT® REV D board using 12 V AC adaptor.

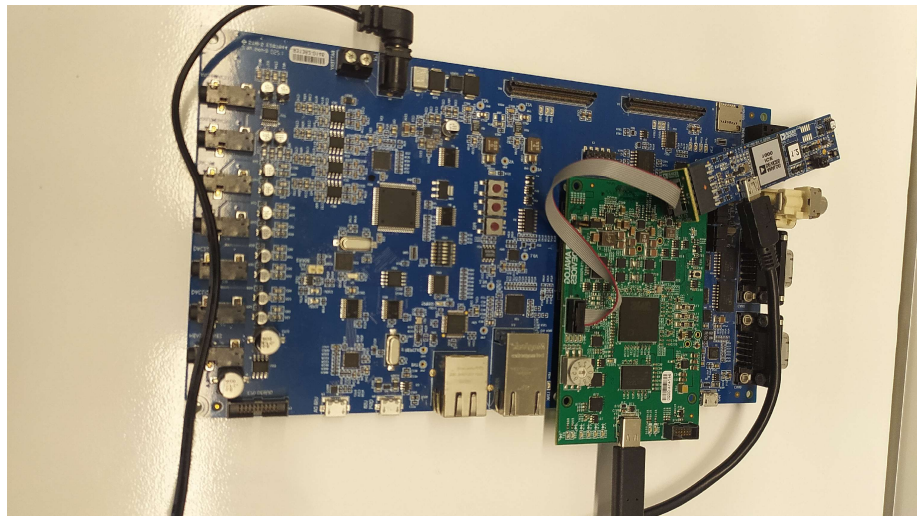


Figure 1: ADSP- SC835 board connections

3.2 Instructions to build and run example application on CCES

3.2.1 To build example projects and run

Follow the steps below to open the project:

- a. The projects can be opened choosing the **File > Import** in CrossCore® Embedded Studio.
- b. In the Import window, select **General > Existing Projects into Workspace** and click **Next**.
- c. In the next window, provide the directory `cces\examples\` from ADI_TFLITE_MICRO_SHARCFX_PROD-Rel4.0.0 package location as **Select root directory** and select any of the projects and click **Finish**. That project will be opened in the IDE's Project Explorer.

Follow the steps below to run the project:

1. If project is not already opened, please open the project as mentioned above.
Right click on the opened **Project** and click on **Build Configurations > Set Active** – choose (Debug/ Release) for the configuration.
2. For the selected configuration:
 - a. Click **Project > Clean**. Ensure that only the required project is selected in the Clean window. If desired, build the project immediately after cleaning. Configure to start a build immediately only for the selected project and click '**OK**'.
 - b. If the previous step has not already built the project, then Click **Project-> Build Project** or press F7.
 - c. The binary (dxe) will be created in the Debug or Release folder located in the workspace depending on the selected configuration. The workspace folder is specified at the top of this section.
3. To debug and run this application using Debug Configuration, please follow the steps documented in CrossCore®EmbeddedStudio_SHARC-FX_Getting_Started_Guide.pdf in section 7 and 7.2(to run with the board) and section 6 (to run with the simulator). This document can be found in Docs folder of your CCES installation. (default location is `C:\analog\cces\3.0.2\Docs`).

3.2.2 Running the File I/O Application

Please follow the readme present in the project folder for more details on how to run and view the outputs. It is recommended to run this with the simulator for getting the outputs faster.

3.2.1 Running the Realtime Application

Please follow the readme present in the project folder for more details on how to run and get the outputs. For audio input/output the connections must be made as shown below

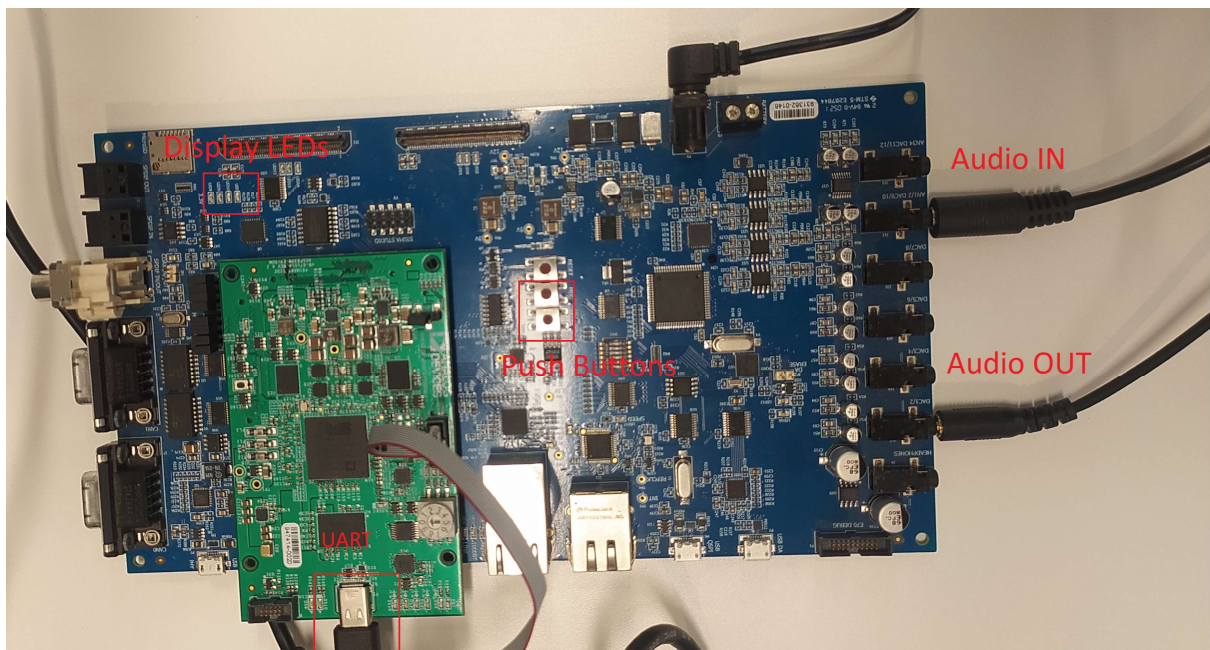


Figure 2: ADSP- SC835 audio connections

3.3 Instructions to build the TFLM library.

The module contains a prebuilt TFLM library with all required optimizations enabled. If you wish to rebuild this with a different configuration, you can follow the steps below-

- Import the adi_sharcfx_nn library (optimized library) project from adi_sharcfx_nn\Project folder.

- You can choose the optimizations you want to enable/disable in the `adi_sharcfx_nn.h` file that is linked in the project.
- Once you choose, you can build this project to get the optimized TFLM library for SharcFX. This library will be generated in '`adi_sharcfx_nn \Lib`' folder.
- Import the TFLM library project from `cces\TFLM\Eagle` folder.
- Build this project to get the library for SharcFX. This TFLM library will be generated in '`cces\TFLM\Lib`' folder.

3.4 Headless Build and Flash Instructions

This project supports headless (command-line) builds, allowing users to compile and flash the application without launching the CCES IDE. The instructions below are platform-agnostic and assume a Unix-like shell environment (e.g., Git Bash on Windows or a Linux terminal).

Note: Always ensure you use the same CCES version to build both the static library and application.

Prerequisites:

- Install Analog Devices CrossCore Embedded Studio (CCES).
- Ensure `make` is accessible in your shell environment.

If not, use:

`/path/to/cces/<version>/make.exe` instead of `make`.

3.4.1 Build Instructions

Navigate to the folder with Makefile and run:

`make`

To specify CCES version explicitly:

`make SHARCFX_ROOT=/path/to/cces/3.0.2`

To clean build artifacts:

`make clean`

To build in Release or Debug mode:

make CONFIG=Release

make CONFIG=Debug

Note: The static library libadi_sharcfx_nn.a and libTFLM.a must be built before compiling the application.

You can find the Makefiles in:

- Optimized TFLM library build: cd adi_sharcfx_nn/Project
- TFLM library build: <root_directory>
- Application build: cd cces/examples/<corresponding_directory>

3.4.2 Flash Instructions

To flash the built application run:

make flash

You may also specify the CCES path:

make flash SHARCFX_ROOT=/path/to/cces/3.0.2

You may also specify the Debugger based on ICE 1000 or ICE 2000:

make flash DEBUGGER=1000

make flash DEBUGGER=2000

Before flashing, make sure to switch to boot mode 0 (No boot) then reset the board. Verify that the application is running by switching to boot mode 1 (SPI boot) then resetting the board again.

3.5 Specifications for current reference applications

Model	Cycle Count	Percentage of DSP used for 1 sec of audio (@1GHz)
Keyword spotter* (@16KHz)	1.359 MCycles	0.14%
Denoiser DTLN* (@16KHz)	1.029 MCycles	3.22%
Genre Identification* (@48KHz)	2.6 GCycles	34%
Urbansound Classification* (@48KHz)	2.6 GCycles	34%
Denoiser DFN* (@48KHz)	1.43 MCycles	18%

(*) Keyword Spotter – 1 forward pass takes in 1 s of audio as input at 16KHz sampling rate.

(*) Denoiser DTLN – 1 forward pass takes in 32 ms of audio as input at 16KHz sampling rate.

(*) Genre Identification – 1 forward pass takes in 2.7 s of audio as input at 48 KHz sampling rate.

(*) Urbansound Classification – 1 forward pass takes in 2.7 s of audio as input at 48 KHz sampling rate.

(*)Denoiser DFN – 1 forward pass takes in 8 ms of audio as input at 48KHz sampling rate.

DFN Licensing Disclaimer:

The Deep Filtering Network (DFN) model is subject to licensing restrictions and cannot be used in commercial products without prior approval. If you require a commercially licensed version of the DFN model, please contact the ADI Eagle-NN team to request a commercially approved model before use.

3.6 UART Setup

- Enable or disable UART_REDIRECT from the realtime application project settings. By default UART_REDIRECT is enabled in realtime application.

- Right Click the realtime project in CCES, select Properties
- Select C/C++ Builds -> Settings -> Preprocessor in SharcFX C/C++ Compiler -> Add or remove UART_REDIRECT
- The output can be directed to the UART in the following way:
 - Make sure to install CP210x USB to UART drivers.
<https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers?tab=downloads>
 - Connect the standard-A end of the USB cable to the host PC and standard-C end to the EV-SC835-SOM Board USB-to-UART connector. Fig 2.
 - Go to Control Panel > Device Manager > Ports (COM & LPT). Ensure that the USB serial port is detected.
 - Determine the COM port used by the USB serial port (for example, COM5).
 - Configure a serial terminal application with the following settings:
 - Bits per second: 9600
 - Data bits: 8
 - Parity: None
 - Stop bits: 1
 - Flow control: None
- Open a serial terminal preferably '**Termite**'. Configure the Termite terminal program with the above settings and also configure it to append the Line feed (LF) for each character sent from the terminal.

3.7 Running your own model (For advanced users)

We have given a few reference applications following the TFLM structure embedded the required functionality in setup and loop functions. You can look at this to understand how to add your model and function.

If you have a custom model which follows a similar preprocessing as in the given reference applications, you can update the header file corresponding to the model with your model header file and the rest of the functionality should run seamlessly.

If you follow a different preprocessing scheme you may have to change this as required in the loop function.

Also to save memory we do not enable all the layers in TFLM. So, if you get a layer not found error, while initializing you can add your layer to

```
static tf::MicroMutableOpResolver<23> micro_op_resolver;
```

4 Flashing application on board using CCES

The steps are described in 2 parts:

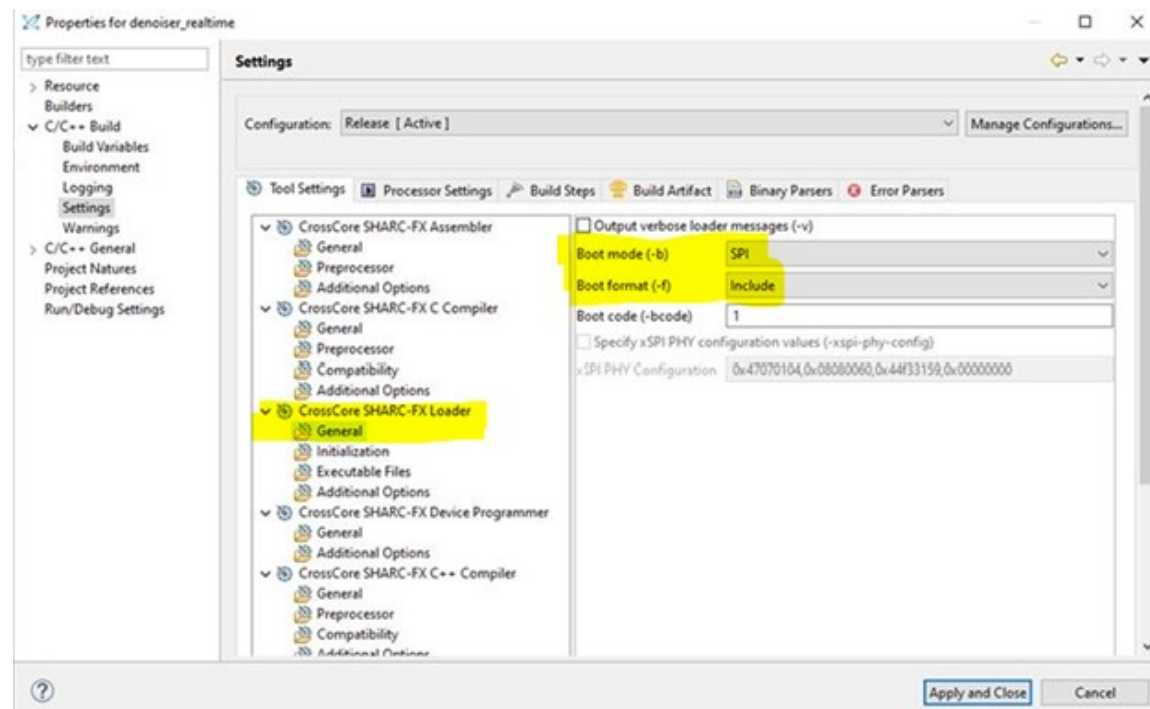
1. Changes to be made in the target application to make the .ldr file boot-ready
2. Flashing the generated loader file onto the board

4.1 Changes to be made in the target application

4.1.1 Ensure boot mode is set to SPI and boot format is set to Include

Boot mode and boot format can be set under:

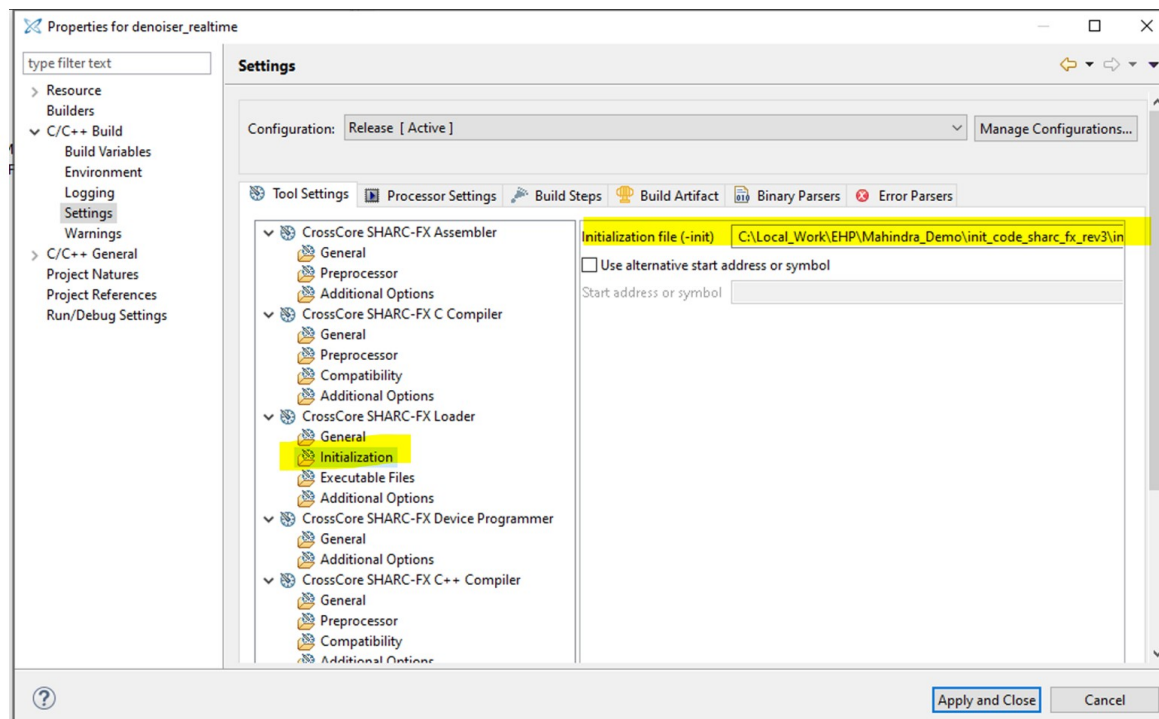
Project properties > C/C++ Build > Settings > Tool Settings > CrossCore SHARC-FX Loader > General



4.1.2 Defining the initialization file

To make the loader file of the target application boot-ready, it needs an initialization sequence which is defined in: `cces \Utils\flashing-tools\init_code_sharc_fx`

Define the init file path under: **Project properties > C/C++ Build > Settings > Tool Settings > CrossCore SHARC-FX Loader > Initialization**



Ensure the entire path to the `init_code_sharc_fx.dxe` file (generated after running the `init_code_sharc_fx` CCES application) is added.

4.1.3 Build target project with these additional settings

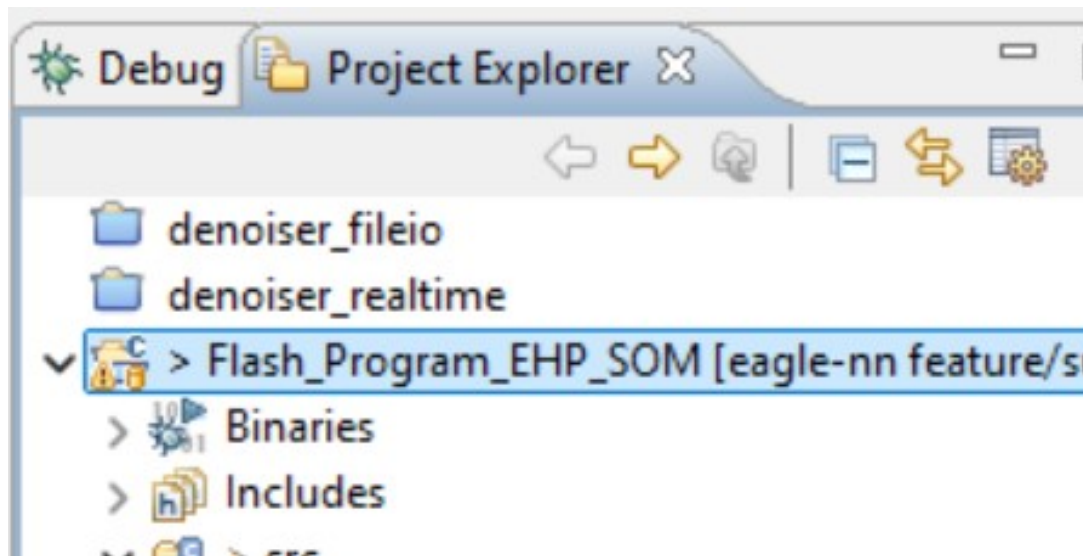
This will generate a loader file (.ldr) which also contains the required initialization sequence.

4.2 Flashing the generated loader file onto the board

4.2.1 Copy the generated loader file into Flash Programmer project

Copy the loader file generated from the previous steps into the src folder of the '*cces \Utils\flashing-tools\Flash_Program_EHP_SOM*' CCES project.

4.2.2 Bring in the loader file into CCES project



By default, the flash programmer will be configured to use the prebuilt loader file for DTLN denoiser application.

This can be changed by replacing it with the path to the loader file you wish to flash onto the board.

4.2.3 Build and Run the Flash Programmer

Build the '*cces \Utils\flashing-tools\Flash_Program_EHP_SOM*' CCES project.

Run/Debug the built project via the **Launch Group configuration** as would be done while loading any other (real-time) application on the hardware. After the run is finished, the target application will be loaded onto the flash. Resetting the board, after this point should boot the board with the target application that was loaded.

4.2.4 Using flash-able prebuilt loader files

There are prebuilt loader files for the `denoiser_realtime`, `genre_id_realtime`, `kws_realtime` and `urbansound_id_realtime`, `denoiser_dfn_realtime` projects that can readily flash-able onto the board. They can be found in the '`cces\prebuilts`'. The prebuilt loader files can be flashed with the same procedure as mentioned above.

5 Multi-Boot Functionality (ADSP-SC83x SPI Flash)

The system supports multi-stage booting, where a bootloader at address 0x0 can load one of several applications stored at different offsets in SPI flash. Application selection is handled at runtime based on user input (e.g., push buttons).

Open the project "`cces\Utils\flashing-tools\bootloader_sharfx`" in CCES and build the project in Release mode.

5.1 Generation Loader file

The headless command for generating loader file from the application dxe.

```
<CCES_path>/elfloader \  
-proc ADSP-SC835 \  
-si-revision 0.0 \  
-bspi \  
-fbinary \  
-bcode 1 \  
-init "C:/analog/cces/3.0.2/Xtensa/SHARC-FX/ldr/ADSPSC835W-EV- SOM_initcode_core1.dxe"  
\  
-core1="<dxe_file>" \  
-verbose \  
-o <output_ldr_file>
```

Note:

- Replace `<dxe_file>` with your actual input dxe file name.
- Replace `<output_ldr_file>` with your actual .ldr file name.
- Before flashing, make sure to switch to boot mode 0 (No boot) then reset the board.
- Verify that the application is running by switching to boot mode 1 (SPI boot) then resetting the board again.

- Use the commands in a UNIX command line tool. The commands were tested using git bash for this release.

5.2 Flashing Overview

Use CLDP (Command Line Device Programmer) to flash the bootloader and applications to their respective offsets. Typical flash layout:

Component	Offset
Bootloader	0x00000000
Denoiser DTLN Application	0x00080000
Genre ID Application	0x00513E00
Urbansound Classification	0x009A7C00
Keyword Spotter Application	0x00E3BA00
Denoiser DFN Application	0x00F00000

Use the following generic format to flash using CLDP:

```
<cces_path>/cldp -proc ADSP-SC835 -emu <1000|2000> -cmd prog -erase affected -
format \ bin -driver <flash_driver_path> -offset <offset> -file <ldr_file>
```

In this release the flash driver path used is

```
<CCES_PATH>/ARM/openocd/share/openocd/scripts/board/flash_algorithms/SC83x_f
lash.dxe
```

Bootloader Behaviour

Upon booting in **SPI Master mode** (boot dial set to 1), the bootloader presents a menu over UART. *(Refer to the UART setup steps in section 3.6 for more details.)*

Hi it's the Bootloader

```
=====
```

SHARC-FX Application Bootloader

=====

Press a key to select an application:

- [1] Denoiser DTLN
- [2] Denoiser DFN
- [3] Genre Identification
- [4] Urban Sound Classification
- [5] Keyword Spotter

--> Auto-booting Denoiser DFN in 10 seconds...

=====

Please follow the “[cces\Utils\flashing-tools\bootloader_sharfx\README.md](#)” for more details.

6 Weights reordering

The new and improved pointwise convolution and fully connected kernels expect model weights to be in non-standard format as it leverages the natural memory access patterns of the DSP to provide better performance. Currently, as of Rel4.0.0, the weights reordering of the model must be done externally to the application code that uses the model and new kernels. The tool to reorder the weights in the format expected by the kernels can be done with the tool provided in the release package. The new kernels are NOT ENABLED out of the box because of this extra step of weights reordering. The procedure to enable them has been illustrated below.

To use the new fully connected and pointwise conv kernels, two main things need to be done:

- Reorder the layer weights
- Enable the new optimized kernels

6.1 Reordering layer weights in the model

The model weights reordering tool found in “cces\Utils\weights-reordering-tool”

The tool takes in a tflite model as input and produces a tflite model as output with the weights of the relevant layers reordered in the formats expected by the respective kernels.

Syntax for tool usage:

```
$python reorder_weights_fc_and_conv1x1.py <input.tflite>  
<output.tflite>
```

The output tflite model can then be converted to the TFLM format and used in the application of choice.

6.2 Enabling new kernels

Currently, the model weights reordering has to be done manually and as such, the new optimizations that expect these reordered weights are not enabled by default. Once the model weights are reordered and added in the application of choice, the new kernels can be enabled by uncommenting the USE_REORDERED_WEIGHTS_SCHEME macro in the “adi_sharcfx_nn.h” header file and the adi_sharcfx_nn lib can be rebuilt to enable the changes.